

REFERAT

- Prognoza Meteo -

Proiect realizat de **Buzenchi Paula-Roberta**, grupa 2B4

Proiectul fost realizat folosind următoarele tehnologii:

- PostgreSQL
- Spring Boot
- React
- PL/pgSQL

Și respectă cerințele proiectului.

1. Cerințele proiectului

Aplicația „Prognoza Meteo” are rolul de a centraliza și analiza informații meteorologice pentru diferite orașe din Europa. Sistemul oferă utilizatorilor prognoze meteo detaliate, statistici, alerte și clasificări ale orașelor pe baza condițiilor

2. Normalizarea bazei de date

Relația inițială

Pentru început, putem considera că toate informațiile aplicației sunt stocate într-o singură relație universală:

R(

country_name,

country_code,

city_name,

latitude,

longitude,

population,

forecast_date,
temperature_min,
temperature_max,
humidity,
wind_speed,
uv_index,
weather_type,

alert_type,
alert_description,
severity,

username,
email,

rating_value,
comment_text
)

Această relație conține informații despre:

- țări;
- orașe;
- prognoze;
- alerte;
- utilizatori;
- comentarii;
- rating-uri.

3. Identificarea dependențelor funcționale

Pe baza cerințelor de business au fost identificate următoarele dependențe funcționale.

A. Dependențe pentru țări

country_name → country_code

country_code → country_name

Codul unei țări identifică unic țara.

B. Dependențe pentru orașe

city_name → latitude, longitude, population, country_name

Un oraș determină:

- coordonatele;
- populația;
- țara asociată.
-

C. Dependențe pentru prognoze

(city_name, forecast_date)

→

**temperature_min,
temperature_max,
humidity,
wind_speed,
uv_index,
weather_type**

Pentru fiecare oraș și zi există o singură prognoză.

Această regulă este implementată și prin constrângerea:

UNIQUE(city_id, date)

din schema bazei de date.

D. Dependențe pentru alerte

(city_name, created_at)

→

**alert_type,
alert_description,
severity**

O alertă este asociată unui anumit oraș și unui moment specific.

E. Dependențe pentru utilizatori

username → email

email → username

Atât username-ul cât și email-ul identifică unic un utilizator.

Acest lucru este impus și prin constrângerile UNIQUE din tabelul users.

F. Dependențe pentru comentarii

(user_id, city_id, created_at) → comment_text

G. Dependențe pentru rating-uri

(user_id, city_id)

→

rating_value

Un utilizator poate oferi un rating pentru un anumit oraș.

4. Identificarea dependențelor multivaluate

În relația universală apar și dependențe multivaluate.

Exemplu:

city_name → alert_type

city_name → comment_text

Un oraș poate avea:

- mai multe alerte;
- mai multe comentarii;
- mai multe rating-uri.

Aceste informații sunt independente între ele și produc redundanță în relația universală.

5. Forma normală 1NF

Pentru a obține forma normală 1NF:

- toate valorile trebuie să fie atomice;
- nu trebuie să existe grupuri repetitive.

Relația respectă 1NF deoarece:

- fiecare câmp conține o singură valoare;
- nu există liste sau attribute multiple în aceeași coloană.

6. Forma normală 2NF

Pentru forma normală 2NF:

- relația trebuie să fie în 1NF;
- toate attributele ne-cheie trebuie să depindă complet de cheia primară.

În relația universală există dependențe parțiale.

Exemplu:

city_name → latitude, longitude, population

Aceste attribute depind doar de city_name, nu de cheia completă:

(city_name, forecast_date)

Prin urmare, relația nu respectă 2NF.

6'. Descompunerea pentru 2NF

Pentru eliminarea dependențelor parțiale, relația a fost descompusă în:

Countries

```
Countries(  
  country_id,  
  country_name,  
  country_code  
)
```

Cities

```
Cities(  
  city_id,  
  city_name,  
  country_id,  
  latitude,  
  longitude,  
  population  
)
```

WeatherForecasts

```
WeatherForecasts(  
  forecast_id,
```

```
city_id,  
forecast_date,  
temperature_min,  
temperature_max,  
humidity,  
wind_speed,  
uv_index,  
weather_type  
)
```

7. Forma normală 3NF

Pentru forma normală 3NF:

- relația trebuie să fie în 2NF;
- nu trebuie să existe dependențe tranzitive.

Exemplu de dependență tranzitivă:

```
city_id → country_id  
country_id → country_name
```

Dacă numele țării ar fi stocat direct în tabela prognozelor, ar apărea redundanță.

Această problemă a fost eliminată prin separarea tabeli `countries`.

8. BCNF (Boyce-Codd Normal Form)

O relație este în BCNF dacă pentru orice dependență funcțională:

$X \rightarrow Y$

atributul X este supercheie.

Tabelele rezultate respectă BCNF deoarece:

- fiecare determinant identifică unic un tuplu;
- nu există dependențe funcționale necontrolate.

Exemple:

```
country_id → country_name
```

city_id → city_name

forecast_id → valorile prognozei

9. Forma normală 4NF

4NF elimină dependențele multivaluate.

În relația universală existau:

city_name → alerts

city_name → comments

city_name → ratings

Aceste dependențe au fost eliminate prin separarea informațiilor în tabele distincte:

- weather_alerts
- comments
- ratings
- weather_anomalies

Astfel:

- redundanța este redusă;
- actualizările devin mai eficiente;
- baza de date respectă 4NF.

10. Diagrama E/A (Entitate-Asociere)

După procesul de normalizare, baza de date a fost împărțită în mai multe entități independente, legate prin relații de tip 1:N sau N:M. Modelul E/A rezultat respectă cerințele aplicației și reduce redundanța datelor.

Entitățile principale:

1. Countries

Entitatea **Countries** stochează informații despre țările existente în sistem.

Atribute:

- id – identificator unic al țării;
- name – numele țării;

- code – codul țării.

Cheie primară:

- id

Constrângeri:

- PRIMARY KEY
- UNIQUE(code)
- NOT NULL(name)

2. Cities

Entitatea Cities stochează informații despre orașe.

Atribute:

- id
- name
- country_id
- latitude
- longitude
- population

Cheie primară:

- id

Cheie externă:

- country_id → countries(id)

Relație:

- o țară poate avea mai multe orașe;
- un oraș aparține unei singure țări.

Cardinalitate:

Countries (1) — (N) Cities

Motivarea cheii externe:

Cheia externă `country_id` a fost introdusă pentru a evita repetarea numelui țării pentru fiecare oraș și pentru a păstra integritatea relațională.

3. WeatherForecasts

Entitatea `WeatherForecasts` stochează prognozele meteorologice.

Atribute:

- `id`
- `city_id`
- `forecast_date`
- `temperature_min`
- `temperature_max`
- `humidity`
- `wind_speed`
- `uv_index`
- `weather_type`

Cheie primară:

- `id`

Cheie externă:

- `city_id → cities(id)`

Relație:

- un oraș poate avea mai multe prognoze;
- o prognoză aparține unui singur oraș.

Cardinalitate:

Cities (1) — (N) WeatherForecasts

Constrângere UNIQUE:

`UNIQUE(city_id, forecast_date)`

Motivarea constrângerii:

Această constrângere împiedică existența mai multor prognoze pentru același oraș și aceeași zi.

4. WeatherAlerts

Entitatea `WeatherAlerts` stochează alertele meteorologice generate automat.

Atribute:

- `id`
- `city_id`
- `alert_type`
- `description`
- `severity`
- `created_at`

Cheie primară:

- `id`

Cheie externă:

- `city_id` → `cities(id)`

Relație:

- un oraș poate avea mai multe alerte;
- o alertă aparține unui singur oraș.

Cardinalitate:

`Cities` (1) — (N) `WeatherAnomalies`

`WeatherForecasts` (1) — (N) `WeatherAnomalies`

6. Users

Entitatea `Users` stochează utilizatorii aplicației.

Atribute:

- `id`
- `username`
- `email`
- `password_hash`
- `created_at`

Cheie primară:

- id

Constrângeri:

UNIQUE(username)

UNIQUE(email)

Motivare:

- fiecare utilizator trebuie identificat unic;
- două conturi nu pot avea același email sau username.

7. Comments

Entitatea Comments stochează comentariile utilizatorilor.

Atribute:

- id
- user_id
- city_id
- comment_text
- created_at

Chei externe:

- user_id → users(id)
- city_id → cities(id)

Relații:

- un utilizator poate adăuga mai multe comentarii;
- un oraș poate avea mai multe comentarii.

Cardinalități:

Users (1) — (N) Comments

Cities (1) — (N) Comments

8. Ratings

Entitatea Ratings stocază evaluările utilizatorilor.

Attribute:

- id
- user_id
- city_id
- rating_value
- created_at

Cheii externe:

- user_id → users(id)
- city_id → cities(id)

Relații:

- un utilizator poate oferi mai multe rating-uri;
- un oraș poate primi mai multe rating-uri.

Cardinalități:

Users (1) — (N) Ratings

Cities (1) — (N) Ratings

Constrângere CHECK:

CHECK(rating_value BETWEEN 1 AND 5)

Motivare:

Rating-ul trebuie să fie între 1 și 5.

9. CityStatistics

Entitatea CityStatistics stocază statistici agregate.

Relație:

Cities (1) — (N) CityStatistics

Un oraș poate avea mai multe statistici generate în timp.

10. CityRankings

Entitatea CityRankings stochează clasamentele generate automat.

Relație:

Cities (1) — (N) CityRankings

Un oraș poate apărea în mai multe clasamente.

11. Asocierile între entități

Relații 1:N

Countries → Cities

O țară conține mai multe orașe.

Cities → WeatherForecasts

Un oraș poate avea mai multe prognoze.

Cities → WeatherAlerts

Un oraș poate avea mai multe alerte.

Cities → WeatherAnomalies

Un oraș poate avea mai multe anomalii.

Users → Comments

Un utilizator poate adăuga mai multe comentarii.

Users → Ratings

Un utilizator poate oferi mai multe rating-uri.

12. Tipuri de constrângeri utilizate

În schema bazei de date au fost utilizate principalele tipuri de constrângeri relaționale.

1. PRIMARY KEY

2. FOREIGN KEY

Exemplu:

FOREIGN KEY (city_id)

REFERENCES cities(id)

3. UNIQUE

Exemplu:

UNIQUE(email)

UNIQUE(username)

UNIQUE(city_id, forecast_date)

4. NOT NULL

Exemplu:

name VARCHAR(100) NOT NULL

5. CHECK

Exemplu:

CHECK(rating_value BETWEEN 1 AND 5)

Motivarea alegerii cheilor

Chei primare

Au fost utilizate câmpuri de tip id pentru:

- identificare unică;
- eficiență în relații;
- indexare rapidă;
- compatibilitate cu relațiile externe.

Chei externe

Cheile externe au fost utilizate pentru:

- eliminarea redundanței;
- menținerea integrității referențiale;
- conectarea corectă a entităților.

13. Scriptul de creare al tabelelor și popularea bazei de date

Baza de date a aplicației „Proгноza Meteo” a fost implementată în PostgreSQL și este alcătuită din mai multe tabele relaționale care gestionează informații despre țări, orașe, prognoze, alerte, utilizatori și statistici.

Pentru identificatorii principali a fost utilizat tipul:

id SERIAL PRIMARY KEY

Acesta permite auto-incrementarea automată a valorilor prin utilizarea secvențelor PostgreSQL.

Exemple de tabele

Tabela Countries

```
CREATE TABLE countries (  
  
    id SERIAL PRIMARY KEY,  
  
    name VARCHAR(100) NOT NULL,  
  
    code VARCHAR(10) UNIQUE  
  
    );
```

Tabela stochează informații despre țări. Câmpul code este unic pentru fiecare țară.

Tabela Cities

```
CREATE TABLE cities(  
  
    id SERIAL PRIMARY KEY,  
  
    name VARCHAR(100) NOT NULL,  
  
    country_id INT NOT NULL,
```

```
CONSTRAINT fk_country  
    FOREIGN KEY (country_id)  
    REFERENCES countries(id)  
    ON DELETE CASCADE  
);
```

Fiecare oraș este asociat unei țări prin cheia externă `country_id`.

Tabela WeatherForecasts

```
CREATE TABLE weather_forecasts (  
    id SERIAL PRIMARY KEY,  
    city_id INT NOT NULL,  
    forecast_date DATE NOT NULL,  
    temperature_min FLOAT,  
    temperature_max FLOAT,  
    humidity FLOAT,  
    wind_speed FLOAT,  
    uv_index FLOAT,  
    weather_type VARCHAR(50)  
);
```

Această tabelă stochează prognozele meteorologice pentru fiecare oraș.

Pentru a evita duplicarea prognozelor a fost utilizată constrângerea:

```
UNIQUE(city_id, forecast_date)
```

Constrângeri utilizate

În baza de date au fost utilizate principalele tipuri de constrângeri:

- PRIMARY KEY
- FOREIGN KEY

- **UNIQUE**
- **NOT NULL**
- **CHECK**

Exemplu:

CHECK(rating_value BETWEEN 1 AND 5)

Această constrângere validează valorile rating-urilor introduse de utilizatori.

Scriptul de populare

Pentru testarea aplicației a fost realizat un script de populare automată a bazei de date.

Acesta introduce:

- țări;
- orașe;
- utilizatori;
- prognoze;
- alerte meteo.

Exemplu:

```
INSERT INTO countries(name, code) VALUES  
('Romania', 'RO'),  
('France', 'FR');
```

Generarea automată a prognozelor

Prognozele sunt generate automat folosind:

generate_series()

și funcția:

random()

Astfel sunt create date meteorologice pentru fiecare zi din an și pentru fiecare oraș.

Proceduri automate

După popularea bazei de date sunt apelate proceduri pentru:

- generarea statisticilor;
- detectarea anomaliilor;
- generarea ranking-urilor.

CALL generate_city_statistics();

CALL detect_weather_anomalies();

Aceste proceduri automatizează procesarea și analiza datelor meteorologice.

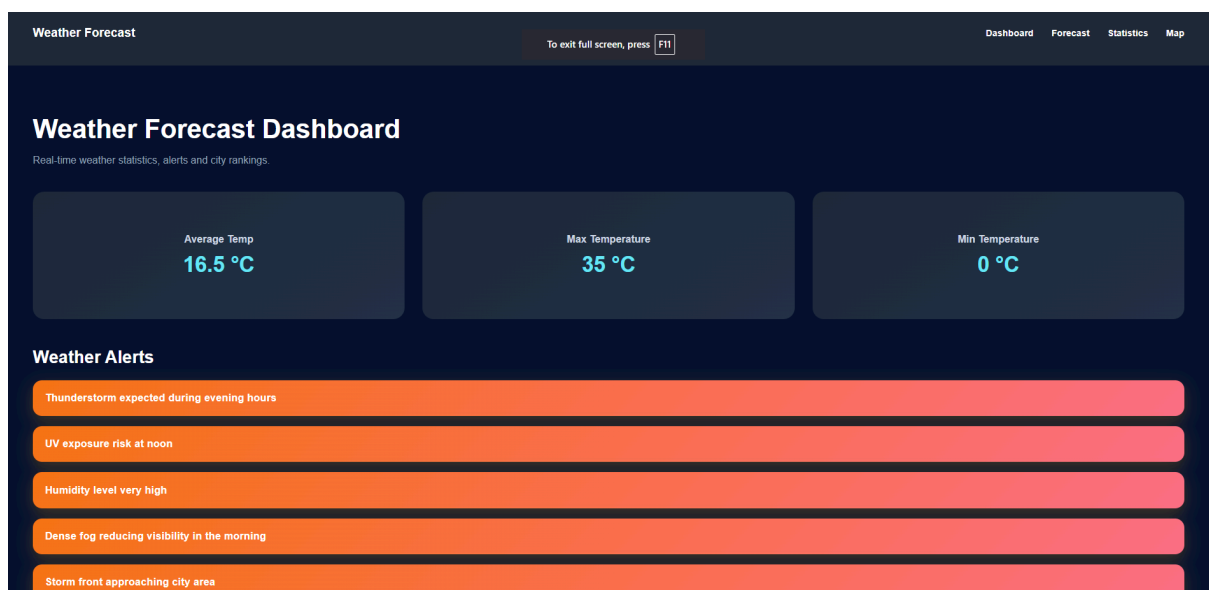
14. Prezentarea aplicației

Interfața principală

Aplicația „Prognoza Meteo” oferă o interfață grafică modernă realizată în React, prin intermediul căreia utilizatorii pot vizualiza prognoze meteorologice, alerte, statistici și clasamente ale orașelor.

În cadrul interfeței sunt afișate:

- prognozele meteo pentru fiecare oraș;
- temperaturile minime și maxime;
- umiditatea;
- viteza vântului;
- indicele UV;
- alertele meteorologice;
- clasamentele orașelor.



High temperature detected

Blizzard conditions possible

Humidity level very high

High temperature detected

Black ice detected on major roads

Continuous rainfall expected for 24 hours

Top Cities

1. Amsterdam16.5°C

2. Oslo16.49°C

3. Bucharest16.45°C

4. Vienna16.43°C

5. Warsaw16.38°C

6. Prague16.38°C

7. Berlin16.35°C

8. Rome16.3°C

9. Paris16.29°C

10. Brussels16.22°C

11. Copenhagen16.19°C

12. Madrid16.17°C

13. Lisbon16.16°C

14. Budapest15.99°C

15. Stockholm15.96°C

Activate Windows
Go to Settings to activate Windows.

Weather Forecast

DashboardForecastStatistics

Weather Statistics

Temperature (Celsius)

100

90

80

70

60

50

40

30

20

10

0

Jan

Feb

Mar

Apr

May

Jun

Jul

Aug

Sep

Oct

Nov

Dec

Hottest City
Amsterdam (16.5°C)

Cooldest City
Stockholm (15.96°C)

Highest Humidity
Budapest (66.47%)

Strongest Wind
Bucharest (12.77 km/h)

Weather Analysis

Amsterdam currently records the highest average temperature.
Stockholm remains the coldest city in the dataset.
Budapest has the highest humidity.
Strong winds are currently detected in Bucharest.

Weather Forecast

DashboardForecastStatistics

Weather Forecast

Explore detailed weather forecasts for major European cities.

CopenhagenNew City choice

2025-01-01

Min Temperature: 8.5°C
Max Temperature: 20.1°C
Wind: 12 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-02

Min Temperature: 9°C
Max Temperature: 20.3°C
Wind: 10 km/h
Humidity: 65%
Weather: ☁️ rain
UV Index: 11

2025-01-03

Min Temperature: 5.4°C
Max Temperature: 20.8°C
Wind: 17 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-04

Min Temperature: 4.9°C
Max Temperature: 21.1°C
Wind: 8 km/h
Humidity: 65%
Weather: ☁️ snow
UV Index: 11

2025-01-05

Min Temperature: 1.3°C
Max Temperature: 20.6°C
Wind: 11 km/h
Humidity: 70%
Weather: ☁️ rain
UV Index: 11

2025-01-06

Min Temperature: 12.8°C
Max Temperature: 19.2°C
Wind: 18 km/h
Humidity: 65%
Weather: ☁️ cloudy
UV Index: 11

2025-01-07

Min Temperature: 3.1°C
Max Temperature: 18.8°C
Wind: 10 km/h
Humidity: 75%
Weather: ☁️ cloudy
UV Index: 11

2025-01-08

Min Temperature: 12.1°C
Max Temperature: 26.8°C
Wind: 16 km/h
Humidity: 75%
Weather: ☀️ sunny
UV Index: 11

2025-01-09

Min Temperature: 13°C
Max Temperature: 19.7°C
Wind: 17 km/h
Humidity: 75%
Weather: ☀️ sunny
UV Index: 11

2025-01-10

Min Temperature: 4.4°C
Max Temperature: 26.8°C
Wind: 7 km/h
Humidity: 75%
Weather: ☁️ rain
UV Index: 11

2025-01-11

Min Temperature: 8.1°C
Max Temperature: 27.7°C
Wind: 17 km/h
Humidity: 65%
Weather: ☁️ cloudy
UV Index: 11

2025-01-12

Min Temperature: 7.4°C
Max Temperature: 26.5°C
Wind: 16 km/h
Humidity: 75%
Weather: ☁️ storm
UV Index: 11

2025-01-13

Min Temperature: 6.6°C
Max Temperature: 18.7°C
Wind: 10 km/h
Humidity: 75%
Weather: ☀️ sunny
UV Index: 11

2025-01-14

Min Temperature: 9°C
Max Temperature: 26.7°C
Wind: 16 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-15

Min Temperature: 5.1°C
Max Temperature: 26.6°C
Wind: 13 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-16

Min Temperature: 13.8°C
Max Temperature: 22°C
Wind: 14 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-17

Min Temperature: 12.7°C
Max Temperature: 26.5°C
Wind: 13 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-18

Min Temperature: 14.9°C
Max Temperature: 26.8°C
Wind: 12 km/h
Humidity: 65%
Weather: ☁️ rain
UV Index: 11

2025-01-19

Min Temperature: 0.8°C
Max Temperature: 24.2°C
Wind: 11 km/h
Humidity: 75%
Weather: ☁️ rain
UV Index: 11

2025-01-20

Min Temperature: 13.2°C
Max Temperature: 19.7°C
Wind: 18 km/h
Humidity: 75%
Weather: ☁️ storm
UV Index: 11

2025-01-21

Min Temperature: 13.1°C
Max Temperature: 19.7°C
Wind: 14 km/h
Humidity: 55%
Weather: ☁️ rain
UV Index: 11

2025-01-22

Min Temperature: 14.4°C
Max Temperature: 16.2°C
Wind: 19 km/h
Humidity: 55%
Weather: ☀️ sunny
UV Index: 11

2025-01-23

Min Temperature: 7.1°C
Max Temperature: 22.2°C
Wind: 16 km/h
Humidity: 65%
Weather: ☁️ cloudy
UV Index: 11

2025-01-24

Min Temperature: 0.6°C
Max Temperature: 26.8°C
Wind: 19 km/h
Humidity: 55%
Weather: ☁️ storm
UV Index: 11

2025-01-25

Min Temperature: 3.8°C
Max Temperature: 16.9°C
Wind: 17 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-26

Min Temperature: 4°C
Max Temperature: 26.1°C
Wind: 14 km/h
Humidity: 55%
Weather: ☁️ rain
UV Index: 11

2025-01-27

Min Temperature: 16.8°C
Max Temperature: 19.8°C
Wind: 13 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-28

Min Temperature: 12.8°C
Max Temperature: 26.8°C
Wind: 12 km/h
Humidity: 65%
Weather: ☁️ cloudy
UV Index: 11

2025-01-29

Min Temperature: 1.4°C
Max Temperature: 25.8°C
Wind: 11 km/h
Humidity: 55%
Weather: ☁️ cloudy
UV Index: 11

2025-01-30

Min Temperature: 14.8°C
Max Temperature: 17.8°C
Wind: 17 km/h
Humidity: 65%
Weather: ☀️ sunny
UV Index: 11

2025-01-31

Min Temperature: 10.1°C
Max Temperature: 23.4°C

2025-02-01

Min Temperature: 4.8°C
Max Temperature: 18°C

2025-02-02

Min Temperature: 10.5°C
Max Temperature: 21.4°C

2025-02-03

Min Temperature: 5.3°C
Max Temperature: 18.4°C

2025-02-04

Min Temperature: 6.4°C
Max Temperature: 21.2°C

2025-02-05

Min Temperature: 7.7°C
Max Temperature: 26.1°C

2025-02-06

Min Temperature: 8.9°C
Max Temperature: 28.4°C

2025-02-07

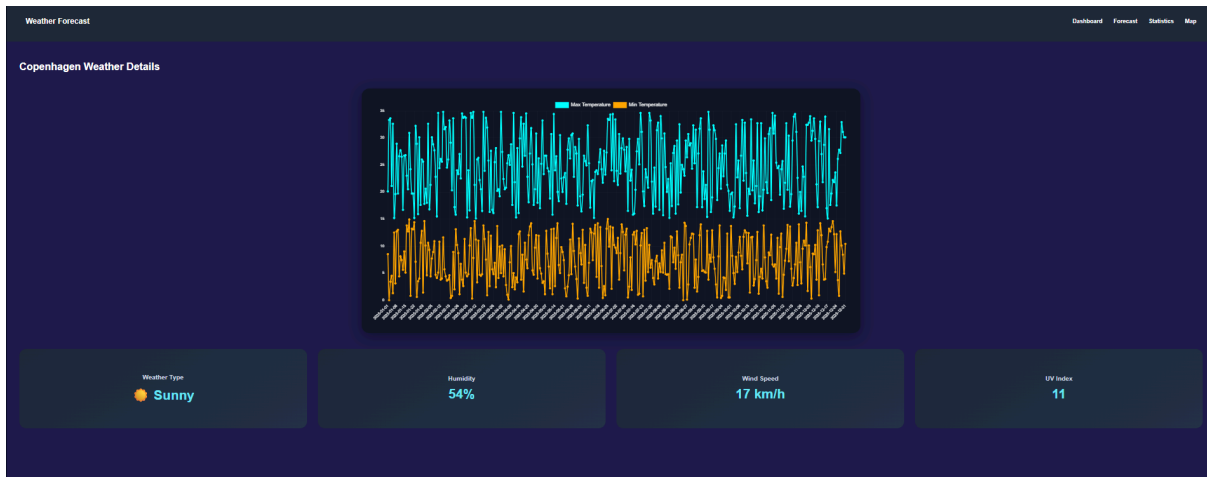
Min Temperature: 10.9°C
Max Temperature: 27.2°C

2025-02-08

Min Temperature: 4.7°C
Max Temperature: 22.4°C

2025-02-09

Min Temperature: 4.1°C
Max Temperature: 13.8°C



15. Explicații pentru algoritmi mai complecși

A. Detectarea anomaliilor meteorologice

Una dintre cele mai importante componente ale aplicației este detectarea automată a anomaliilor meteorologice.

Această funcționalitate este implementată prin procedura:

`detect_weather_anomalies()`

Procedura analizează valorile prognozelor și identifică:

- temperaturi extreme;
- umiditate anormală;
- viteze foarte mari ale vântului;
- diferențe semnificative față de valorile medii.

Rezultatele sunt salvate în tabela:

`weather_anomalies`

și pot fi afișate ulterior în aplicație.

B. Generarea statisticilor

Statisticile meteorologice sunt generate automat folosind procedura:

`generate_city_statistics()`

Aceasta calculează:

- temperatura medie;

- temperatura minimă și maximă;
- media umidității;
- media vitezei vântului.

Valorile sunt calculate pe baza tuturor prognozelor existente pentru fiecare oraș și sunt salvate în tabela `city_statistics`.

C. Generarea ranking-urilor

Aplicația poate genera clasamente automate folosind procedura:

`generate_city_rankings()`

Aceasta realizează:

- topul celor mai calde orașe;
- topul celor mai reci orașe;
- clasificări bazate pe diferite valori meteorologice.

Rezultatele sunt salvate în tabela:

`city_rankings`

și sunt afișate în frontend sub formă de liste și statistici.

Generarea automată a datelor

Pentru popularea bazei de date a fost utilizată funcția:

`generate_series()`

împreună cu:

`random()`

Acestea permit generarea automată a prognozelor pentru fiecare zi din an și pentru fiecare oraș, facilitând testarea aplicației și realizarea statisticilor.

16. Concluzii

În cadrul acestui proiect a fost realizată aplicația „Prognoza Meteo”, o platformă full-stack pentru gestionarea și analiza informațiilor meteorologice. Aplicația permite stocarea prognozelor, generarea alertelor, calcularea statisticilor și detectarea anomaliilor meteorologice.

Baza de date a fost proiectată și normalizată pentru a elimina redundanțele și pentru a asigura integritatea datelor, utilizând chei primare, chei externe și constrângeri specifice. De asemenea, au fost implementate proceduri și funcții PL/pgSQL pentru automatizarea procesării datelor.

Proiectul a oferit aplicarea practică a conceptelor studiate la baza de date și integrarea acestora într-o aplicație completă realizată cu PostgreSQL, Spring Boot și React.